

PHP

(Laravel, Symfony)

Guia para crear API-Rest Laravel

Fase 1

1. Verificar el entorno; instalación de php, composer, mysql.
 - a. `php --version`
 - b. `composer --version`
 - c. `mysql --version`
2. Crear el proyecto con composer, generar la clave secreta, configurar la base de datos en el .env, crear la base de datos en phpmyadmin, realizar la primera migración y correr el servidor.
 - a. `composer create-project laravel/laravel "nombre carpeta del proyecto"`
 - b. `cd "nombre carpeta del proyecto" - php artisan key:generate`
 - c. descomentar las líneas de configuración de la base de datos en el .env y agregar los datos de la conexión.
 - d. ir a phpmyadmin y crear la base de datos.
 - e. `php artisan migrate`
 - f. `php artisan serve`

Fase 2

1. Crear las tablas necesarias para el proyecto con sus respectivas migraciones, editar los archivos de las tablas creados en database/migrations en la función up y hacer la migración.
 - a. Para cada una de las tablas: `php artisan make:model create_"nombreTabla"_table --create="nombreTabla"`
 - b. En la función up de cada migración agregar los campos necesarios con `$table->TipoDato('nombreCampo');`
 - c. `php artisan migrate`
2. crear los modelos de las tablas, editar el archivo de cada modelo generado en App/Models/ agregando los fillable y si algún dato necesita casting hacerlo.
 - a. `php artisan make:model "NombreTabla"`
 - b. Agregar los protected fillable para inserción masiva con `protected $fillable = [ 'campos'... ]`
 - c. Los datos que necesiten casting se necesita crear un protected `$casts = [ 'campos'... ]`
3. hacer una comprobación rápida de las tablas usando Tinker (Intérprete interactivo de laravel).
 - a. `php artisan tinker` y dentro hacer una creación en cada tabla creada para comprobar su funcionamiento. `nombreModelo::create(['campo' => 'dato'...]);` y `nombreModelo::all()->toArray();` para que la salida sea un array tipo json.

Fase 3

1. Crear los controladores necesarios y editar cada archivo generado en `app/Http/Controllers/Api/` en cada una de sus funciones `index()`, `store()`, `show()`, `update()`, `destroy()`.
 - a. `php artisan make:controller Api/NombreController -api`
 - b. `public function index()`
 - c. `{`
 - d. `return response()->json(Model::all());`
 - e. `}`
 - f.
 - g. `public function store(Request $request)`
 - h. `{`
 - i. `$validated = $request->validate([`
 - j. `'campo1' => 'required|string|max:255',`
 - k. `'campo2' => 'nullable|string',`
 - l. `]);`
 - m.
 - n. `$variable = Model::create($validated);`
 - o. `return response()->json($variable, 201);`
 - p. `}`
 - q.
 - r. `public function show(Model $variable)`
 - s. `{`
 - t. `return response()->json($variable);`
 - u. `}`
 - v.
 - w. `public function update(Request $request, Model $variable)`
 - x. `{`
 - y. `$validated = $request->validate([`
 - z. `'campo1' => 'sometimes|required|string|max:255',`
 - aa. `'campo2' => 'nullable|string',`
 - bb. `'campo3' => 'boolean',`
 - cc. `]);`
 - dd.
 - ee. `$variable->update($validated);`
 - ff. `return response()->json($variable);`
 - gg. `}`
 - hh.
 - ii. `public function destroy(Model $variable)`
 - jj. `{`
 - kk. `$variable->delete();`
 - ll. `return response()->json(null, 204);`
 - mm. `}`

2. crear en routes/ el archivo api.php
 - a. use Illuminate\Support\Facades\Route;
 - b. use App\Http\Controllers\Api\NombreController;
 - c.
 - d. Route::apiResource('nombre en plural', NombreController::class);
 - e.
3. correr el servidor y comprobar en <http://127.0.0.1:8000/api/principal>
 - a. php artisan serve

Fase 4

1. crear un request personalizado y editar su contenido en el archivo generado en app/Http/Resources/"Nombre"Resource.php.
2. crear un resource y editar su contenido en el archivo generado en app/Http/Resources/"Nombre"Resource.php.
3. Actualizar el controlador principal y editar su contenido en el archivo en app/Http/Controllers/Api/"Nombre"Controller.php.
4. correr el servidor y comprobar en <http://127.0.0.1:8000/api/principal>

Fase 5

1. Crear el frontend en public/frontend/index.html - [app.js](#) -styles.css
 - a. debo crear la carpeta frontend y los archivos.
 - b. <form id="form-creacion">
 - c. <input type="text" name="titulo" id="titulo">
 - d. <input type="text" name="descripcion" id="descripcion">
 - e. <button type="submit">Crear</button>
 - f. </form>
 - g.
 - h. <ul id="lista">
 - i. <script>
 - j. const API_URL = "http://127.0.0.1:8000/api/principal" // "principal"
puede cambiar por el enfoque principal de la API
 - k.
 - l. const formCreacion = document.getElementById("form-creacion");
 - m. const lista = document.getElementById("lista");
 - n.
 - o. async function fetchDatos() {
 - p. const respuesta = await fetch(API_URL);
 - q. const datos = await respuesta.json();
 - r. renderizarDatos(datos);
 - s. }
 - t.
 - u. function renderizarDatos(datos) {
 - v. datos.forEach(dato => {
 - w. const elemento = document.createElement("li");

```

x.
y.      const titulo = document.createElement("span");
z.      titulo.classList.add(`${datos.completada ? "completada": ""}`);
aa.     titulo.textContent = `${datos.titulo}`;
bb.
cc.     elemento.appendChild(titulo);
dd.
ee.     const contenedor = document.createElement("div");
ff.
gg.     const botonGestiona = document.createElement("button");
hh.     botonGestiona.onclick = `gestionaData(${dato.id},
    ${dato.completada})`;
ii.     botonGestiona.textContent = `${dato.completada ? "Reabrir":
    "Completar"}`;
jj.     contenedor.appendChild(botonGestiona);
kk.
ll.     const botonElimina = document.createElement("button");
mm.     botonElimina.onclick = `eliminarDato${dato.id}`;
nn.     botonElimina.textContent = "Eliminar";
oo.     contenedor.appendChild(botonElimina);
pp.
qq.     elemento.appendChild(contenedor);
rr.
ss.     lista.appendChild(elemento);
tt.   })
uu.   }
vv.
ww.     formCreacion.addEventListener("submit", async e => {
xx.     e.preventDefault();
yy.
zz.     const titulo = document.getElementById("titulo").value;
aaa.     const descripcion =
        document.getElementById("descripcion").value;
bbb.
ccc.     await fetch(API_URL, {
ddd.         method: "POST",
eee.         headers: {
fff.             "Content-Type": "application/json"
ggg.         },
hhh.         body: JSON.stringify({titulo, descripcion}),
iii.     });
jjj.     formCreacion.reset();
kkk.     fetchDatos();
lll.   });
mmm.

```

```

nnn.      async function gestionaDato(id, completada) {
ooo.          await fetch(`${API_URL}/${id}`, {
ppp.              method: "PATCH",
qqq.              headers: {
rrr.                  "Content-Type": "application/json"
sss.              },
ttt.          body: JSON.stringify({completada: !completada}),
uuu.          });
vvv.          fetchDatos();
www.      }

xxx.
yyy.      async function eliminarDato(id) {
zzz.          await fetch(`${API_URL}/${id}`, {
aaaa.              method: "DELETE"
bbbb.          });
cccc.          fetchDatos();
dddd.      }
eeee.      </script>

```